



**AMRITA VISHWA VIDYAPEETHAM  
COIMBATORE CAMPUS  
TIFAC-CORE In Cyber Security**

**20CYS403 – Web Application Security**

**Course Instructor : Dr. Jevitha K P**

**PHP Programming Lab**

## **Learning Objectives**

At the end of this laboratory session, students will be able to:

1. Configure and test a PHP development environment.
2. Write and execute basic PHP programs.
3. Use variables, operators, and control statements.
4. Develop programs using arrays and functions.
5. Process HTML forms using PHP.
6. Perform basic server-side validation.
7. Create and read cookies for client-side state management.
8. Handle sessions for user state management.
9. Connect PHP applications to a MySQL database.
10. Insert and retrieve records from MySQL using PHP.

## **Software Requirements**

- XAMPP/WAMP/LAMP Stack
- Apache Web Server
- PHP 8.x
- MySQL/MariaDB
- phpMyAdmin
- VS Code / Visual Studio Code
- Modern Web Browser (Chrome/Firefox)

# Folder Structure

Create a folder inside the web server directory.

```
htdocs/  
  php_lab/  
    hello.php  
    variables.php  
    operators.php  
    grade.php  
    loops.php  
    arrays.php  
    functions.php  
    form.html  
    process.php  
    calculator.php  
    cookie.php  
    view_cookie.php  
    session.php  
    view_session.php  
    login.php  
    dashboard.php  
    logout.php  
    db_connect.php  
    insert_student.php  
    create_table.sql
```

Each file or pair of files in this folder is used for one exercise in the lab. Keeping the files separate makes it easier for beginners to understand one concept at a time.

Start Apache and MySQL from XAMPP and open:

[http://localhost/php\\_lab/](http://localhost/php_lab/)

# Exercise 1 – Creating Your First PHP Program

## Objective

Understand PHP syntax and execution.

## Step 1

Create a file named:

hello.php

## Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello PHP</title>
</head>
<body>

<?php
// echo is used to display output on the web page
echo "<h2>Welcome to PHP Programming Lab</h2>";
echo "Hello World!";
?>

</body>
</html>
```

## Code Explanation

- <!DOCTYPE html> tells the browser that this is an HTML5 document.
- <html>, <head>, and <body> are standard HTML tags.
- <?php ... ?> is the PHP code block.
- echo prints text or HTML content to the browser.
- The first echo displays a heading.
- The second echo displays a simple message.

## Expected Output

Welcome to PHP Programming Lab

Hello World!

## Note

PHP code runs on the server before the page is sent to the browser. The browser only sees the final output, not the PHP code itself.

# Exercise 2 – Variables and Data Types

## Objective

Learn variables and string interpolation.

Create:

variables.php

## Code

**<?php**

```
// String variable  
$name = "Alice";
```

```
// Integer variable  
$age = 20;
```

```
// Float variable  
$cgpa = 8.75;
```

```
// Boolean variable  
$isStudent = true;
```

```
echo "Name: $name <br>";  
echo "Age: $age <br>";  
echo "CGPA: $cgpa <br>";
```

```
// Boolean values are often used in conditions  
if($isStudent)  
{  
    echo "Status: Student";  
}
```

**?>**

## Code Explanation

- \$name, \$age, \$cgpa, and \$isStudent are variables.
- PHP variable names always begin with \$.
- "Alice" is a string.
- 20 is an integer.
- 8.75 is a floating-point number.
- true is a boolean value.
- echo "Name: \$name" shows how PHP can directly insert variable values inside double quotes.
- <br> adds a line break in HTML.

- The if statement checks whether \$isStudent is true.

## Task

Modify the program to display your own details.

## Note

Variables are used to store data temporarily while the program is running. They are one of the most important parts of programming.

## Exercise 3 – Operators

Create:

operators.php

Code

```
<?php
```

```
$a = 15;
```

```
$b = 4;
```

```
echo "Addition = ".$a+$b."<br>";
```

```
echo "Subtraction = ".$a-$b."<br>";
```

```
echo "Multiplication = ".$a*$b."<br>";
```

```
echo "Division = ".$a/$b."<br>";
```

```
echo "Modulus = ".$a%$b."<br>";
```

```
?>
```

### Code Explanation

- \$a and \$b store two numbers.
- + is the addition operator.
- - is the subtraction operator.
- \* is the multiplication operator.
- / is the division operator.
- % gives the remainder after division, which is called modulus.
- . is the string concatenation operator in PHP. It joins text and values together.

### Activity

Change the values of \$a and \$b.

Observe the output.

### Note

Operators help perform calculations and comparisons in PHP programs.

## Exercise 4 – Conditional Statements

Create:

grade.php

Code

```
<?php
```

```
$marks = 82;
```

```
if($marks >= 90)
{
    echo "Grade A";
}
elseif($marks >= 75)
{
    echo "Grade B";
}
elseif($marks >= 60)
{
    echo "Grade C";
}
else
{
    echo "Fail";
}
```

```
?>
```

### Code Explanation

- \$marks stores the student's score.
- if checks the first condition.
- elseif checks another condition if the previous one is false.
- else runs when none of the conditions are true.
- The program assigns a grade based on the marks.

### Exercise

Test with different marks.

### Note

Conditional statements allow a program to make decisions based on data.

## Exercise 5 – Loops

Create:

loops.php

Code

```
<?php
```

```
echo "<h3>For Loop</h3>";
```

```
// The for loop is used when the number of repetitions is known
```

```
for($i = 1; $i <= 10; $i++)
```

```
{
```

```
    echo $i."<br>";
```

```
}
```

```
echo "<h3>While Loop</h3>";
```

```
// The while loop repeats as long as the condition is true
```

```
$j = 1;
```

```
while($j <= 5)
```

```
{
```

```
    echo $j."<br>";
```

```
    $j++;
```

```
}
```

```
?>
```

### Code Explanation

- A for loop is useful when you know how many times you want to repeat something.
- `$i = 1` initializes the counter.
- `$i <= 10` is the condition.
- `$i++` increases the counter by 1 after each loop.
- A while loop continues until its condition becomes false.
- `$j++` is necessary to avoid an infinite loop.

### Activity

Display the multiplication table of 7.

### Note

Loops are used to repeat tasks without writing the same code again and again.



## Exercise 6 – Arrays

Create:

arrays.php

Code

```
<?php
```

```
$subjects = array(
    "Web Programming",
    "Operating Systems",
    "Networks",
    "Cyber Security"
);

echo "<h3>Subjects</h3>";

//foreach is used to read each item in an array one by one
foreach($subjects as $subject)
{
    echo $subject."<br>";
}

?>
```

### Code Explanation

- An array stores multiple values in a single variable.
- \$subjects contains a list of subject names.
- foreach is the easiest way to read all values in an array.
- \$subject represents one item at a time from the array.

### Challenge

Store five student names in an array and print them.

### Note

Arrays are useful when you need to store a collection of related values.

## Exercise 7 – Cookies

Create:

cookie.php

Code

```
<?php
// setcookie() stores data in the browser for a limited time
setcookie("student_name", "Rahul", time() + 3600, "/");
setcookie("course", "PHP Programming", time() + 3600, "/");

echo "Cookie variables set successfully.<br>";
echo '<a href="view_cookie.php">View Cookie Data</a>';
?>
```

Create:

view\_cookie.php

Code

```
<?php

echo "<h2>Cookie Data</h2>";

if(isset($_COOKIE["student_name"]))
{
    echo "Student Name: ".$_COOKIE["student_name"]."<br>";
}
else
{
    echo "Student Name cookie not set.<br>";
}

if(isset($_COOKIE["course"]))
{
    echo "Course: ".$_COOKIE["course"]."<br>";
}
else
{
    echo "Course cookie not set.<br>";
}

?>
```

Code Explanation

- setcookie() creates a cookie and sends it to the browser.
- time() + 3600 sets the cookie to expire after one hour.

- The / path makes the cookie available throughout the application.
- \$\_COOKIE is a PHP superglobal used to read cookie values.
- isset() checks whether the cookie exists before trying to display it.

## Activity

Change the cookie values and expiry time. Try setting the expiry time to a past value to delete the cookie.

## Note

Cookies are stored in the browser, not on the server. They are useful for remembering preferences, but sensitive data should not be stored in them.

## Exercise 8 – User Defined Functions

Create:

functions.php

Code

**<?php**

*// This function calculates the square of a number*

```
function square($number)
{
    return $number * $number;
}
```

```
echo "Square of 6 = ".square(6);
```

**?>**

### Code Explanation

- A function is a reusable block of code.
- square(\$number) is a function that accepts one input.
- return sends the result back to the caller.
- square(6) calls the function with the value 6.

### Exercise

Create another function to calculate the area of a rectangle.

### Note

Functions help reduce repetition and make programs easier to read and maintain.

## Exercise 9 – HTML Forms

Create:

form.html

### Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Form</title>
</head>
<body>

  <!-- The form sends data to process.php using the POST method -->
  <form action="process.php" method="post">

    Name:
    <input type="text" name="name">

    <br><br>

    Age:
    <input type="number" name="age">

    <br><br>

    <input type="submit" value="Submit">
  </form>

</body>
</html>
```

### Code Explanation

- <form> creates a form for user input.
- action="process.php" means the form data will be sent to process.php.
- method="post" sends data in the request body, which is better for form submissions.
- name="name" and name="age" are important because PHP uses these names to read the submitted values.
- The submit button sends the form data.

### Note

HTML forms are used to collect information from users and send it to the server.

## Exercise 10 – Form Processing

Create:

process.php

Code

```
<?php
```

```
// Read values sent from the HTML form
```

```
$name = $_POST["name"];
```

```
$age = $_POST["age"];
```

```
echo "<h2>Student Details</h2>";
```

```
echo "Name : ".$name."<br>";
```

```
echo "Age : ".$age;
```

```
?>
```

### Code Explanation

- \$\_POST is a PHP superglobal array that stores form data sent using the POST method.
- \$\_POST["name"] gets the value entered in the Name field.
- \$\_POST["age"] gets the value entered in the Age field.
- echo displays the submitted values on the page.

### Activity

Submit different values and observe the output.

### Note

Form processing means reading user input on the server and using it in the program.

# Exercise 11 – Simple Calculator

Create:

calculator.php

## Code

```
<!DOCTYPE html>
<html>
<body>

<form method="post">

    Number 1:
    <input type="number" name="n1">

    <br><br>

    Number 2:
    <input type="number" name="n2">

    <br><br>

    <input type="submit" value="Calculate">
</form>

<?php

// Check whether the form has been submitted
if($_SERVER["REQUEST_METHOD"] == "POST")
{
    $n1 = $_POST["n1"];
    $n2 = $_POST["n2"];

    echo "<h3>Results</h3>";

    echo "Addition = " . ($n1 + $n2) . "<br>";
    echo "Subtraction = " . ($n1 - $n2) . "<br>";
    echo "Multiplication = " . ($n1 * $n2) . "<br>";

    // Division by zero must be avoided
    if($n2 != 0)
    {
        echo "Division = " . ($n1 / $n2);
    }
    else
    {
        echo "Division by zero not allowed";
    }
}
```

```
}  
?>  
  
</body>  
</html>
```

### Code Explanation

- The form collects two numbers from the user.
- `$_SERVER["REQUEST_METHOD"] == "POST"` checks whether the form has been submitted.
- The program performs addition, subtraction, multiplication, and division.
- The `if($n2 != 0)` condition prevents division by zero, which would cause an error.

### Note

This exercise shows how PHP can combine HTML forms and calculations in one file.



## Exercise 12 – Basic Form Validation

Update **process.php**

Code

**<?php**

```
$name = trim($_POST["name"]);
$age = $_POST["age"];

// empty() checks whether the value is blank
if(empty($name))
{
    die("Name cannot be empty");
}

// Simple age check
if($age < 18)
{
    echo "Minor Student";
}
else
{
    echo "Adult Student";
}

?>
```

### Code Explanation

- trim() removes extra spaces from the beginning and end of the input.
- empty(\$name) checks whether the name field is blank.
- die() stops the program immediately and displays the message.
- The age check classifies the student as minor or adult.

### Discussion

Why is server-side validation important even when client-side validation is used?

### Note

Client-side validation can be bypassed, so server-side validation is necessary to protect the application.

# Exercise 13 – Session Handling Basics

## Objective

Understand how to create and access session variables.

Create:

session.php

## Code

```
<?php
// session_start() must be called before using session variables
session_start();

$_SESSION["student_name"] = "Rahul";
$_SESSION["course"] = "PHP Programming";

echo "Session variables set successfully.<br>";
echo "Student Name: " . $_SESSION["student_name"] . "<br>";
echo "Course: " . $_SESSION["course"] . "<br>";

echo '<a href="view_session.php">View Session Data</a>';
?>
```

Create:

view\_session.php

## Code

```
<?php
session_start();

echo "<h2>Session Data</h2>";
echo "Student Name: " . $_SESSION["student_name"] . "<br>";
echo "Course: " . $_SESSION["course"] . "<br>";
?>
```

## Code Explanation

- A session stores user data on the server.
- session\_start() begins or resumes a session.
- \$\_SESSION["student\_name"] stores the student name.
- \$\_SESSION["course"] stores the course name.
- The second file reads the same session values and displays them.

## Activity

Refresh the page and observe that session data remains available until the session is destroyed.

## Note

Sessions are commonly used to remember user information across multiple pages. Unlike cookies, session data is stored on the server and is generally more secure for sensitive information.

# Exercise 14 – Session-Based Login and Logout

## Objective

Develop a simple login system using sessions.

Create:

login.php

## Code

```
<?php
session_start();

if(isset($_POST["login"]))
{
    $username = $_POST["username"];
    $password = $_POST["password"];

    // Simple hardcoded login check for learning purposes
    if($username === "admin" && $password === "1234")
    {
        $_SESSION["username"] = $username;
        header("Location: dashboard.php");
        exit();
    }
    else
    {
        $error = "Invalid username or password";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>

<h2>Login Form</h2>

<?php
if(isset($error))
{
    echo "<p style='color:red;'>$error</p>";
}
?>

<form method="post">
```

Username:

```
<input type="text" name="username">
<br><br>
```

Password:

```
<input type="password" name="password">
<br><br>
```

```
<input type="submit" name="login" value="Login">
</form>
```

```
</body>
```

```
</html>
```

Create:

dashboard.php

Code

```
<?php
```

```
session_start();
```

```
// If the user is not logged in, send them back to the login page
```

```
if(!isset($_SESSION["username"]))
```

```
{
```

```
    header("Location: login.php");
```

```
    exit();
```

```
}
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Dashboard</title>
```

```
</head>
```

```
<body>
```

```
<h2>Welcome, <?php echo $_SESSION["username"]; ?></h2>
```

```
<a href="logout.php">Logout</a>
```

```
</body>
```

```
</html>
```

Create:

logout.php

## Code

```
<?php
session_start();

// Remove all session variables
session_unset();

// Destroy the session completely
session_destroy();

header("Location: login.php");
exit();
?>
```

## Code Explanation

- login.php checks whether the submitted username and password match the expected values.
- If the login is successful, the username is stored in the session.
- header("Location: dashboard.php") redirects the user to the dashboard.
- dashboard.php checks whether the session exists before showing the page.
- logout.php clears and destroys the session so the user is logged out.

## Activity

Change the username and password values and test the login flow again.

## Note

This is a basic example of authentication using sessions. In real applications, passwords should never be hardcoded, and secure password hashing should be used instead.

# Exercise 15 – Connecting PHP to MySQL

## Objective

Establish a connection between PHP and MySQL.

Create a database and table using phpMyAdmin or SQL.

create\_table.sql

## Code

```
CREATE DATABASE php_lab;
```

```
USE php_lab;
```

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    age INT NOT NULL  
);
```

## Code Explanation

- **CREATE DATABASE** php\_lab; creates a new database.
- **USE** php\_lab; selects the database for use.
- **CREATE TABLE** students creates a table named students.
- id is the primary key and increases automatically for each new record.
- name, email, and age store student information.

Create:

db\_connect.php

## Code

```
<?php
```

```
// Connect to MySQL server
```

```
$conn = mysqli_connect("localhost", "root", "", "php_lab");
```

```
// Check whether the connection was successful
```

```
if(!$conn)
```

```
{
```

```
    die("Connection failed: " . mysqli_connect_error();
```

```
}
```

```
echo "Connected to MySQL successfully";
```

```
?>
```

## Code Explanation

- `mysqli_connect()` connects PHP to the MySQL server.
- `"localhost"` means the database is on the same computer.
- `"root"` is the default MySQL username in XAMPP.
- `""` is the password field, which is empty by default in many local setups.
- `"php_lab"` is the database name.
- `mysqli_connect_error()` gives the reason if the connection fails.

## Activity

Change the database name or password if required and test the connection.

## Note

Before reading or writing data in a database, PHP must first connect to MySQL.



# Exercise 16 – Insert and Display Records from MySQL

## Objective

Insert student records into MySQL and display them in a table.

Create:

insert\_student.php

## Code

**<?php**

```
$conn = mysqli_connect("localhost", "root", "", "php_lab");

if(!$conn)
{
    die("Connection failed: " . mysqli_connect_error());
}

if($_SERVER["REQUEST_METHOD"] == "POST")
{
    $name = trim($_POST["name"]);
    $email = trim($_POST["email"]);
    $age = (int)$_POST["age"];

    // Prepared statement helps safely insert data into the database
    $stmt = mysqli_prepare($conn, "INSERT INTO students (name, email, age) VALUES (?, ?, ?)");
    mysqli_stmt_bind_param($stmt, "ssi", $name, $email, $age);
    mysqli_stmt_execute($stmt);

    echo "<p style='color:green;'>Student record inserted successfully.</p>";

    mysqli_stmt_close($stmt);
}

// Fetch all student records from the database
$result = mysqli_query($conn, "SELECT * FROM students");
?>

<!DOCTYPE html>
<html>
<head>
    <title>Student Records</title>
</head>
<body>

<h2>Insert Student Record</h2>
```

```

<form method="post">
    Name:
    <input type="text" name="name">
    <br><br>

    Email:
    <input type="email" name="email">
    <br><br>

    Age:
    <input type="number" name="age">
    <br><br>

    <input type="submit" value="Save">
</form>

<h2>Student List</h2>

<table border="1" cellpadding="8">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Age</th>
    </tr>

    <?php
    while($row = mysqli_fetch_assoc($result))
    {
        echo "<tr>";
        echo "<td>".$row["id"]."</td>";
        echo "<td>".$row["name"]."</td>";
        echo "<td>".$row["email"]."</td>";
        echo "<td>".$row["age"]."</td>";
        echo "</tr>";
    }
    ?>
</table>

</body>
</html>

```

## Code Explanation

- The script first connects to the database.
- When the form is submitted, the entered values are read using \$\_POST.
- trim() removes extra spaces from text fields.
- (int) converts the age value into an integer.
- mysqli\_prepare() creates a prepared statement, which is a safer way to insert data.

- `mysqli_stmt_bind_param()` attaches the values to the query.
- `mysqli_stmt_execute()` runs the insert query.
- `mysqli_query()` retrieves all rows from the students table.
- `mysqli_fetch_assoc()` reads one row at a time as an associative array.
- The while loop displays each record in the HTML table.

## Activity

Insert at least three student records and verify that they appear in the table.

## Note

Prepared statements are recommended because they make database operations safer and more reliable.

## Mini Challenge (20 Minutes)

Develop a **Student Grade Calculator**.

The program should:

- Accept student name.
- Accept marks in three subjects.
- Compute total marks.
- Compute average.
- Display grade using if-else conditions.

### Sample Output

Student Name : Rahul

Total : 246

Average : 82

Grade : B

To complete this challenge:

1. Create a form to accept the student name and three marks.
2. Use PHP to read the submitted values.
3. Add the marks to calculate the total.
4. Divide the total by 3 to calculate the average.
5. Use if-else statements to assign a grade.

## Extension Activity

Enhance the grade calculator to:

- Display Pass/Fail status.
- Show Highest Mark.
- Show Lowest Mark.
- Store subject marks in an array.
- Use functions for calculating total and average.

This extension helps you practice:

- Arrays for storing marks.
- Functions for reusable calculations.
- Conditional statements for result classification.

## Lab Deliverables

Students should submit a document with :

1. Source code of all exercises.
2. Screenshots of outputs.
3. Student Grade Calculator.